

Rapport sur le projet de sfsd

1. Introduction

Ce document fournit une analyse des structures de données et des algorithmes implémentés dans le programme "META.c". Le programme gère un système de stockage utilisant des métadonnées, des blocs mémoire et des fichiers. Les différentes opérations incluent l'insertion, la suppression, la recherche, la défragmentation et la gestion globale des fichiers.

2. Structures de données

2.1 Structure ENREGISTREMENT

- **Description** : Cette structure représente un enregistrement de données.
- **Champs** :
 - int ID: Identifiant unique de l'enregistrement.
 - char champs[60]: Données associées à l'enregistrement.
 - int deleted: Indique si l'enregistrement est supprimé (logiquement).

2.2 Structure Bloc

- **Description** : Chaque bloc contient un ensemble d'enregistrements et des informations sur son état.
- **Champs** :
 - int is_occupied: Statut d'occupation du bloc.
 - int nbrEnreg: Nombre d'enregistrements présents dans le bloc.
 - int next_bloc: Adresse du bloc suivant (utile pour les fichiers chaînés).
 - ENREGISTREMENT enreg[FB]: Tableau d'enregistrements.
 - int adresse_bloc: Adresse physique du bloc.

2.3 Structure META

- **Description** : Contient les métadonnées associées à un fichier.

- **Champs :**
 - char nom_fichier[50]: Nom du fichier.
 - int taille_blocs: Nombre de blocs utilisés par le fichier.
 - int taille_enregistrements: Nombre total d'enregistrements dans le fichier.
 - int adresse_premier_bloc: Adresse du premier bloc du fichier.
 - int organisation_globale: Mode d'organisation globale (contigu ou chaîné).
 - int organisation_interne: Mode d'organisation interne (trié ou non trié).

2.4 Structure File

- **Description :** Représente un fichier et ses blocs associés.
- **Champs :**
 - META meta: Métadonnées associées au fichier.
 - Bloc blocs[MAX_BLOCS]: Tableau de blocs.

2.5 Tableau d'allocation

- **Description :** Utilisé pour suivre l'état d'occupation des blocs.
- **Définition :**
 - Bloc* allocation_table: Tableau contenant l'état d'occupation des blocs (libre ou occupé).

3. Algorithmes

3.1 Initialisation

- **Fonction:** initialize_disk
- **Description:** Initialise le système de stockage en marquant tous les blocs comme libres et prépare le tableau d'allocation.
- **Complexité:**
 - Temps: $O(n)$, où n est le nombre de blocs.

3.2 Création de fichier

- **Fonction:** `creer_fichier`
- **Description:** Crée un nouveau fichier avec les métadonnées fournies par l'utilisateur et alloue les blocs requis.
- **Détails:**
 - Vérifie l'espace libre.
 - Configure les métadonnées (nom, organisation, taille).
 - Alloue les blocs et configure leurs liens (pour les fichiers chaînés).
- **Complexité:**
 - Temps: $O(b)$, où b est le nombre de blocs requis.

3.3 Insertion d'enregistrements

- **Fonctions:**
 - `insertion_contigue_non_triee`
 - `insertion_contigue_triee`
 - `insertionChaineNonTrie`
 - `insertion_chaine_triee`
- **Description:** Insèrent des enregistrements selon l'organisation interne et globale du fichier.
- **Détails:**
 - Les insertions triées impliquent un décalage pour maintenir l'ordre.
 - Les insertions dans les fichiers chaînés peuvent nécessiter la création de nouveaux blocs.
- **Complexité:**
 - Trié: $O(m + \log(n))$, où m est le nombre de blocs et n est le nombre d'enregistrements dans un bloc.
 - Non trié: $O(m)$.

3.4 Suppression

- **Fonctions:**
 - `suppression_logique`
 - `suppression_physique`
- **Description:** Supprime un enregistrement de manière logique ou physique.
- **Détails:**
 - Logique: Marque l'enregistrement comme supprimé.
 - Physique: Supprime et décale les enregistrements restants.
- **Complexité:**
 - Logique: $O(n)$.
 - Physique: $O(n + m)$.

3.5 Recherche

- **Fonctions:**
 - `recherche_contigue_triee`
 - `recherche_contigue_non_triee`
 - `rechercheChaineTrie`
 - `rechercheChaineNonTrie`
- **Description:** Recherchent un enregistrement selon l'organisation des fichiers.
- **Détails:**
 - Utilisent des recherches linéaires ou binaires.
 - Les fichiers triés permettent une recherche plus rapide.
- **Complexité:**
 - Trié: $O(\log(n) + m)$.
 - Non trié: $O(n + m)$.

3.6 Compactage et défragmentation

- **Fonctions:**

- compact
- défragmentation

- **Description:**

- Compactage: Réorganise les blocs pour optimiser l'espace.
- Défragmentation: Rassemble les enregistrements valides dans des blocs contigus.

- **Complexité:**

- $O(m + b)$, où m est le nombre d'enregistrements et b est le nombre de blocs.

4. Conclusion

Ce programme met en œuvre des structures et des algorithmes bien conçus pour gérer efficacement la mémoire secondaire. Les mécanismes d'organisation et de gestion garantissent une utilisation optimale des ressources tout en permettant une flexibilité dans les opérations sur les fichiers.